

Title Page

LAB 7: DNS Introduction and Traffic Analysis

Student Name: Athanasius Orikeze Alekwe

Student ID: 2025/FWSD/11230.

Course Name: Computer and Digital Forensics

Instructor Name: Aminu Idris

Date of Submission: 22/09/2026

Executive Summary

In this lab, I examined normal DNS activity so I could understand how domain names are resolved and how DNS traffic can be used during a forensic investigation. I first identified the DNS resolver configured on my Kali Linux system, then used dig to query A, AAAA, MX and NS records for example.com. I also captured a fresh DNS query and analysed the request and response fields, including the transaction ID, client and resolver addresses, ports, response code, returned IP addresses, TTL values and response timing.

I then generated browser-based DNS traffic by using Firefox Private Browsing to access Google and perform a search. This produced a wider range of DNS requests for Google, Mozilla, fonts, advertising and other supporting services. I later created a separate browser correlation capture so I could compare DNS answers with subsequent TCP connections. One clear result showed ssl.gstatic.com resolving to 142.251.216.35, followed shortly afterwards by a TCP connection to the same address on port 443. I also completed the optional SMTP-DNS analysis using my Lab 4 evidence, where mail.patriots.in ultimately resolved to 74.53.140.153. These activities gave me a practical view of normal DNS behaviour and how DNS evidence can be linked with later network activity.

Resolver and Environment

I carried out the practical in a Kali Linux virtual machine running in VMware Workstation. Before starting the DNS analysis, I checked that the required tools were available on the system, including dig, TShark and Wireshark. I also created the Lab 7 directory structure so that evidence, working files, reports, screenshots, exports and scripts were stored separately during the exercise.

I checked /etc/resolv.conf and confirmed that my configured DNS resolver was:

192.168.45.2

This resolver was later seen again in the fresh DNS capture, which helped me confirm that the traffic I analysed was being sent to the DNS server configured on the system. For the fresh query, my client address was 192.168.45.130, using source port 55556, while the resolver used UDP port 53. The DNS transaction ID was 0xb763, and the request for example.com returned 104.20.23.154 and 172.66.147.243.

I also confirmed that eth0 was the correct capture interface before collecting the DNS evidence. During the lab, I preserved the authorised training PCAP and created a working copy. The original and working copies produced the same SHA-256 hash, which gave me confidence that the working evidence had not changed from the preserved source file.

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/Lab Identifier	SBT-DF203-Lab7-Athanasius-Orikeze-Alekwe
Trainee Name	Athanasius Orikeze Alekwe
Student ID	2025/FWSD/11230
Date Started	21 September 2026
Analysis Workstation / VM	Kali Linux 2025.3 running in VMware Workstation
Evidence File(s)	dig_dns.pcap, dig_dns_working.pcap, fresh_dig_dns.pcapng, browser_dns.pcapng, browser_correlation.pcapng, and Lab 4 smtp_working.pcap used for the optional SMTP correlation
Source / Generation Method	dig_dns.pcap was downloaded from the authorised training repository. fresh_dig_dns.pcapng was generated from a controlled dig +noedns example.com A query. browser_dns.pcapng was generated while browsing Google. browser_correlation.pcapng was generated during browser activity to retain both DNS and later TCP traffic.
Original SHA-256	dig_dns.pcap: 9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d
Working-Copy SHA-256	dig_dns_working.pcap: 9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d
Fresh Capture SHA-256	fresh_dig_dns.pcapng: 665660c8417943f395b542dd3fdabac58e9d6fc2dba2188d72bd130f35371a
Integrity Result	The original training PCAP and working copy produced identical SHA-256 values, confirming that the working copy matched the preserved source evidence.

Field	Student Entry
Notes on Changes	Original evidence was preserved. Analysis was performed on working or newly generated captures. An additional browser_correlation.pcapng was created because the DNS-only browser capture did not contain the later TCP packets required for connection correlation.

Theory Questions and Explanations

What is DNS?

DNS, or the Domain Name System, translates human-readable domain names into IP addresses that computers use for communication. It allows users to work with names such as example.com instead of needing to remember numerical IP addresses.

The practical demonstrates this directly: example.com was queried and returned IPv4 and IPv6 addresses.

What is recursive DNS resolution?

Recursive resolution occurs when a client asks a DNS resolver for the final answer to a DNS query. The resolver takes responsibility for obtaining the requested information, whether from its own cache or by querying other DNS servers.

In this lab, the Kali client sent its DNS requests to the configured resolver at:

192.168.45.2

The resolver then returned the requested DNS answers.

What is iterative DNS resolution?

In iterative resolution, one DNS server may not provide the final answer. Instead, it can direct the requester toward another DNS server that is closer to the authoritative source.

For example, DNS resolution may progress through:

Root server → TLD server → authoritative name server

until the requested record is found.

What is a DNS resolver?

A DNS resolver receives requests from a client and helps locate the requested DNS information.

In this practical, the configured resolver was:

192.168.45.2

The fresh query was sent from the Kali client to this address on UDP port 53.

What does IN mean in a DNS record?

IN means Internet.

It identifies the DNS record as belonging to the Internet namespace/class.

For example:

example.com. 5 IN A 104.20.23.154

uses IN as the DNS class.

Lab Folder Structure and Evidence Preparation

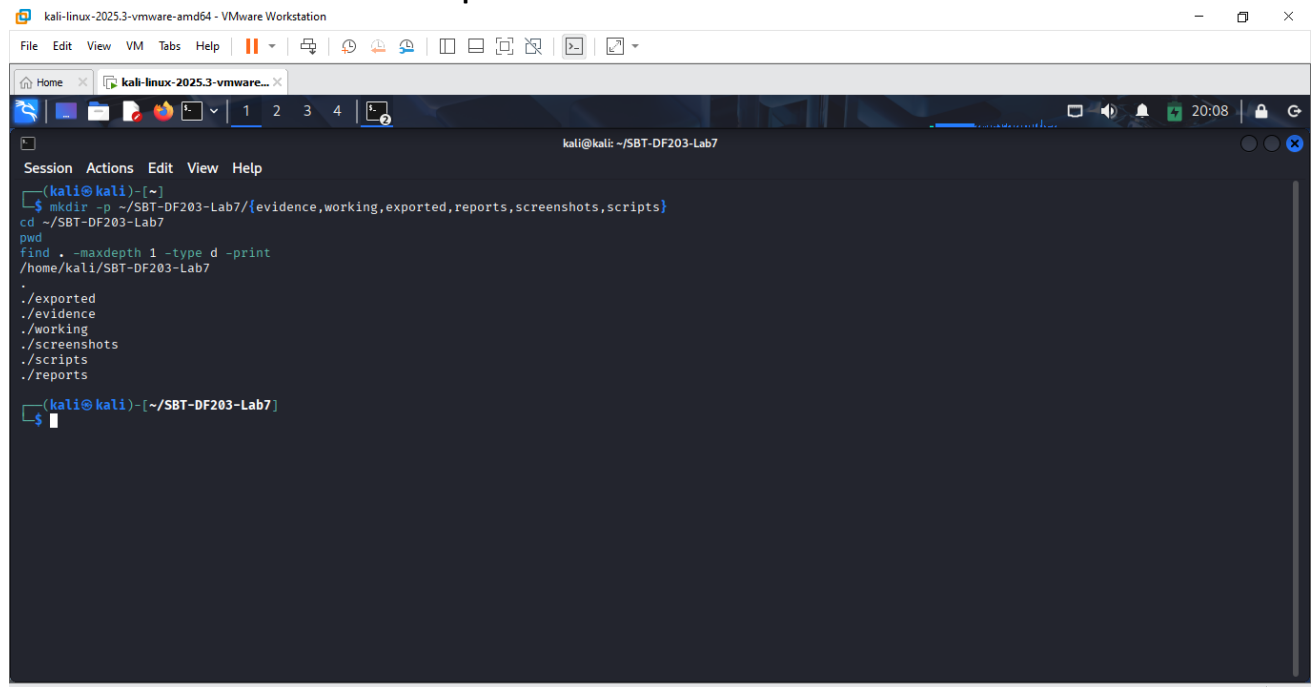


Fig 1.0 Creating the Lab folder.

Explanation

The screenshot shows the creation of the Lab 7 working directory and the separate folders used for evidence, reports, screenshots, scripts, exports and working files.

Keeping the evidence and analysis files in separate locations helped maintain an organized forensic workflow. The folder structure was created before evidence collection so the practical work could be stored consistently throughout the lab.

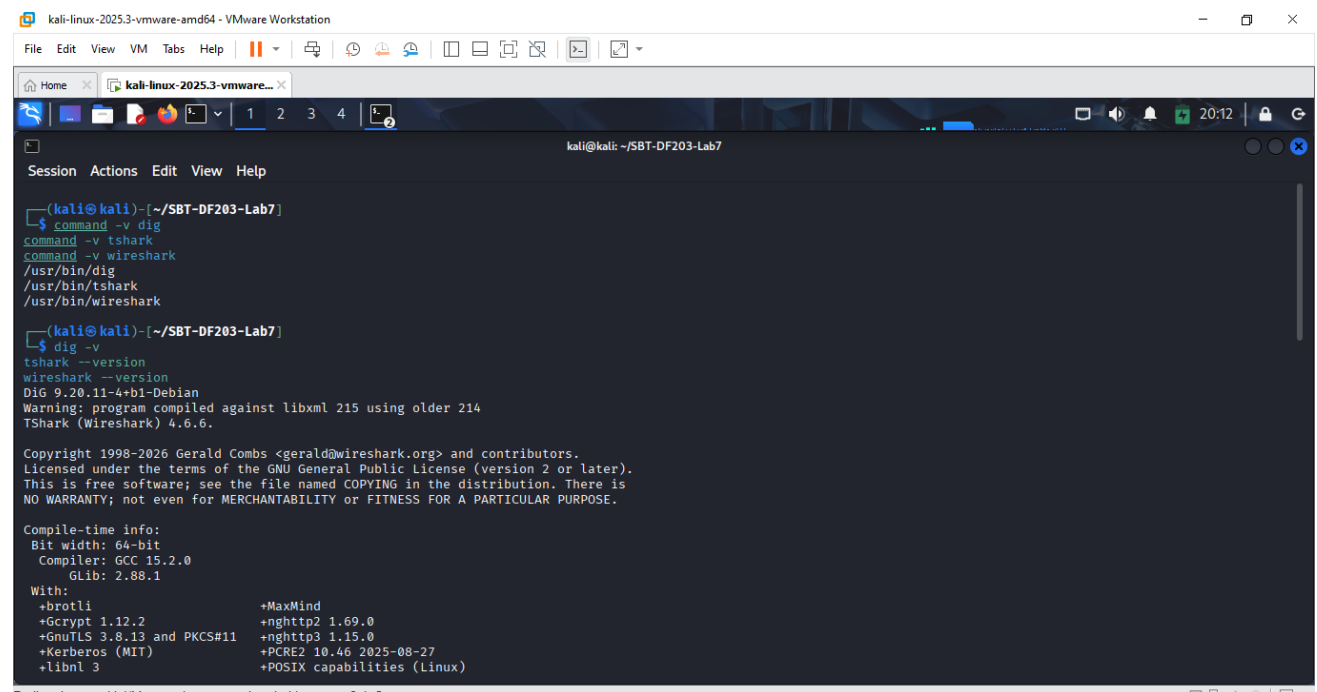


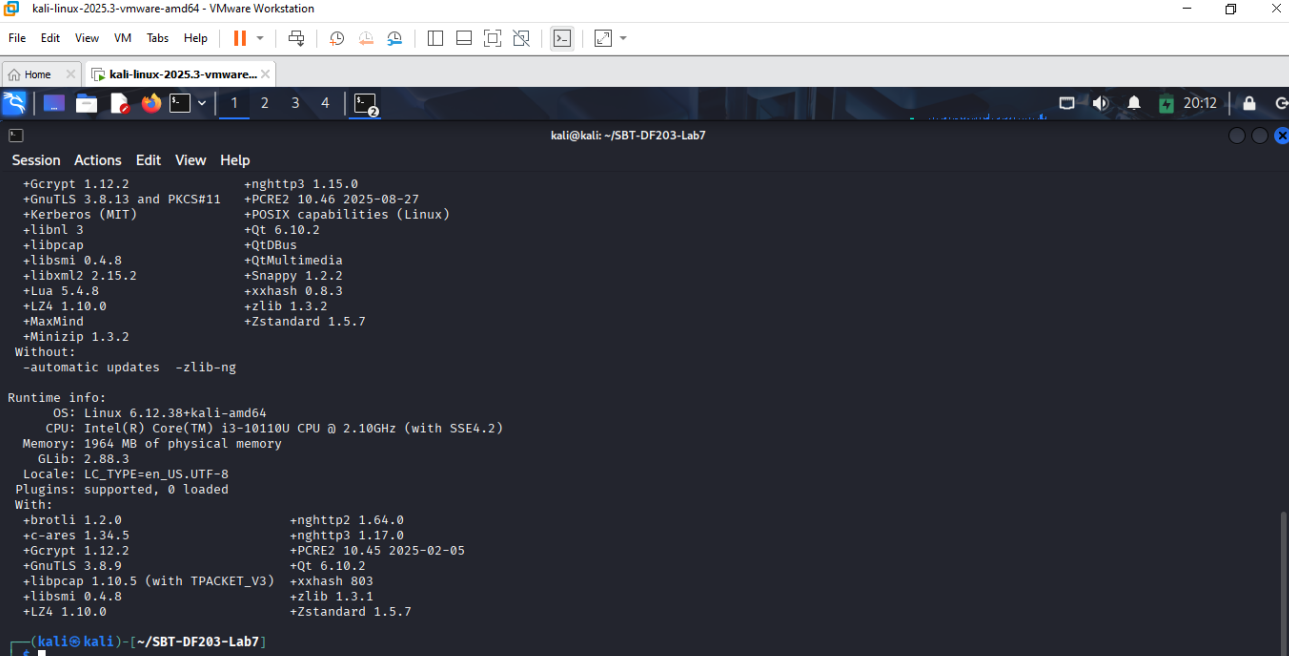
Fig 1.1 Checking the required tools.

Explanation

This screenshot confirms that the main DNS and packet-analysis tools needed for the practical were available on the Kali Linux system.

The terminal identified dig, tshark and Wireshark, showing that the required applications could be used without reinstalling them.

The displayed versions also provided a record of the software environment used during the analysis.



The screenshot shows a terminal window titled 'kali@kali: ~/SBT-DF203-Lab7'. The terminal output lists installed packages and their versions, organized into two columns. The first column lists packages like Gcrypt, GnuTLS, Kerberos, libnl, libpcap, libsmi, libxml2, Lua, LZ4, MaxMind, and Minizip. The second column lists packages like nghttp3, PCRE2, POSIX capabilities, Qt, QtDBus, QtMultimedia, Snappy, xxhash, zlib, and Zstandard. Below the package list, the terminal shows runtime information including the OS (Linux 6.12.38+kali-amd64), CPU (Intel(R) Core(TM) i3-10110U), memory (1964 MB), locale (LC_TYPE=en_US.UTF-8), and plugins (supported, 0 loaded). The terminal prompt is '(kali@kali)~\$'.

```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
+Gcrypt 1.12.2      +nghttp3 1.15.0
+GnuTLS 3.8.13 and PKCS#11 +PCRE2 10.46 2025-08-27
+Kerberos (MIT)    +POSIX capabilities (Linux)
+libnl 3           +Qt 6.10.2
+libpcap           +QtDBus
+libsmi 0.4.8      +QtMultimedia
+libxml2 2.15.2    +Snappy 1.2.2
+Lua 5.4.8         +xxhash 0.8.3
+LZ4 1.10.0        +zlib 1.3.2
+MaxMind           +Zstandard 1.5.7
+Minizip 1.3.2
Without:
-automagic updates -zlib-ng

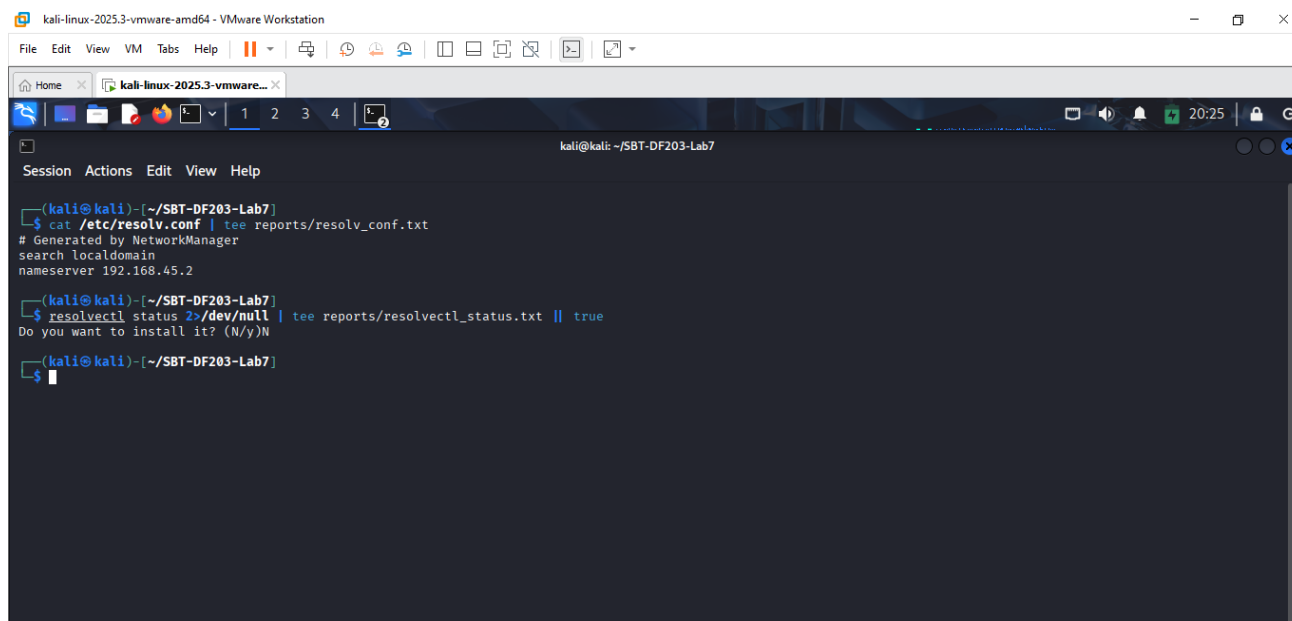
Runtime info:
OS: Linux 6.12.38+kali-amd64
CPU: Intel(R) Core(TM) i3-10110U CPU @ 2.10GHz (with SSE4.2)
Memory: 1964 MB of physical memory
Glib: 2.88.3
Locale: LC_TYPE=en_US.UTF-8
Plugins: supported, 0 loaded
With:
+brotli 1.2.0      +nghttp2 1.64.0
+c-ares 1.34.5     +nghttp3 1.17.0
+Gcrypt 1.12.2     +PCRE2 10.45 2025-02-05
+GnuTLS 3.8.9      +Qt 6.10.2
+libpcap 1.10.5 (with TPACKET_V3) +xxhash 803
+libsmi 0.4.8      +zlib 1.3.1
+LZ4 1.10.0        +Zstandard 1.5.7

(kali@kali)~$
```

Fig 1.2.1 Checking the required tools page 1.

Explanation

The second tool-check screenshot continues the version information for the installed packet-analysis software. It confirms that TShark/Wireshark loaded successfully together with their supporting libraries. This gave confidence that the workstation was ready for DNS packet capture and examination.



```
kali@kali: ~/SBT-DF203-Lab7
$ cat /etc/resolv.conf | tee reports/resolv_conf.txt
# Generated by NetworkManager
search localdomain
nameserver 192.168.45.2

(kali@kali)~/SBT-DF203-Lab7
$ resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true
Do you want to install it? (N/y)N

(kali@kali)~/SBT-DF203-Lab7
$
```

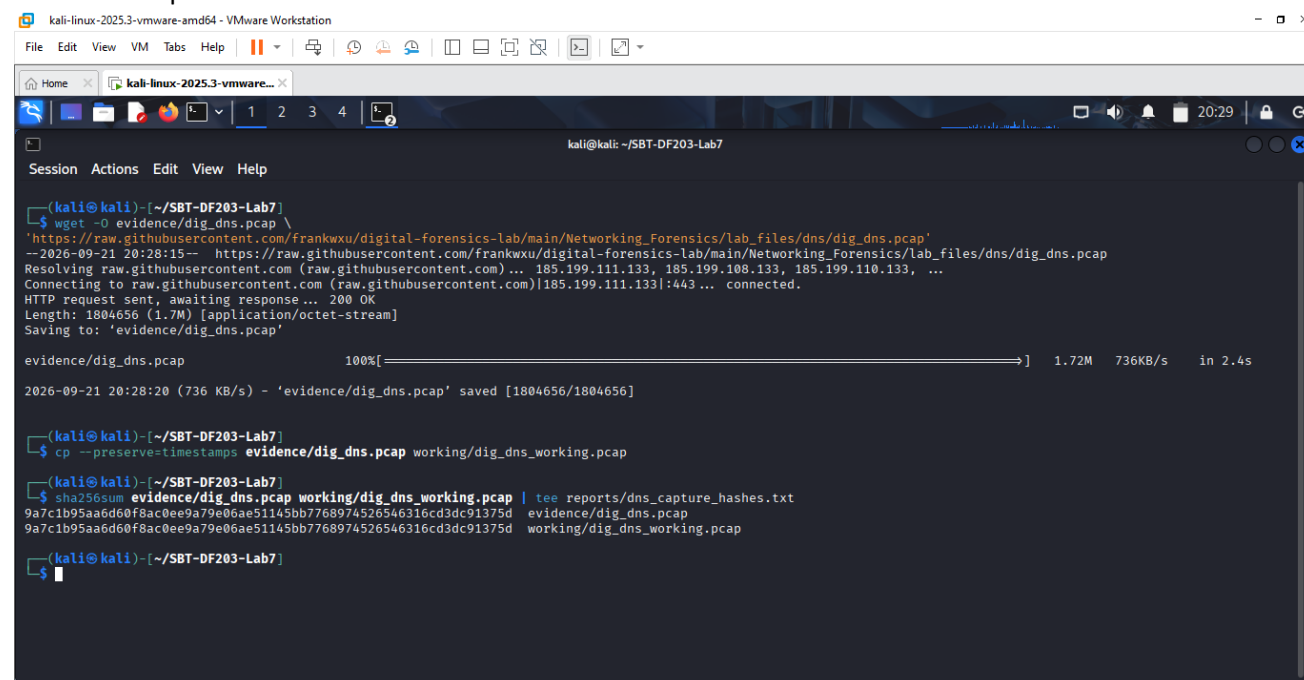
Fig 1.3 Configured DNS Resolver.

Explanation

The resolver configuration showed that the Kali system was using 192.168.45.2 for DNS name resolution.

This value was obtained from /etc/resolv.conf before generating fresh DNS traffic.

Recording the resolver first made it possible to later confirm that captured DNS requests were actually being sent to the expected server.



```
kali@kali: ~/SBT-DF203-Lab7
$ wget -O evidence/dig_dns.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
--2026-09-21 20:28:15-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com) ... 185.199.111.133, 185.199.108.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1804656 (1.7M) [application/octet-stream]
Saving to: 'evidence/dig_dns.pcap'

evidence/dig_dns.pcap      100%[=====] 1.72M  736KB/s  in 2.4s

2026-09-21 20:28:20 (736 KB/s) - 'evidence/dig_dns.pcap' saved [1804656/1804656]

(kali@kali)~/SBT-DF203-Lab7
$ cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap

(kali@kali)~/SBT-DF203-Lab7
$ sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  evidence/dig_dns.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  working/dig_dns_working.pcap

(kali@kali)~/SBT-DF203-Lab7
$
```

Fig 1.4 Evidence preparation / hash verification.

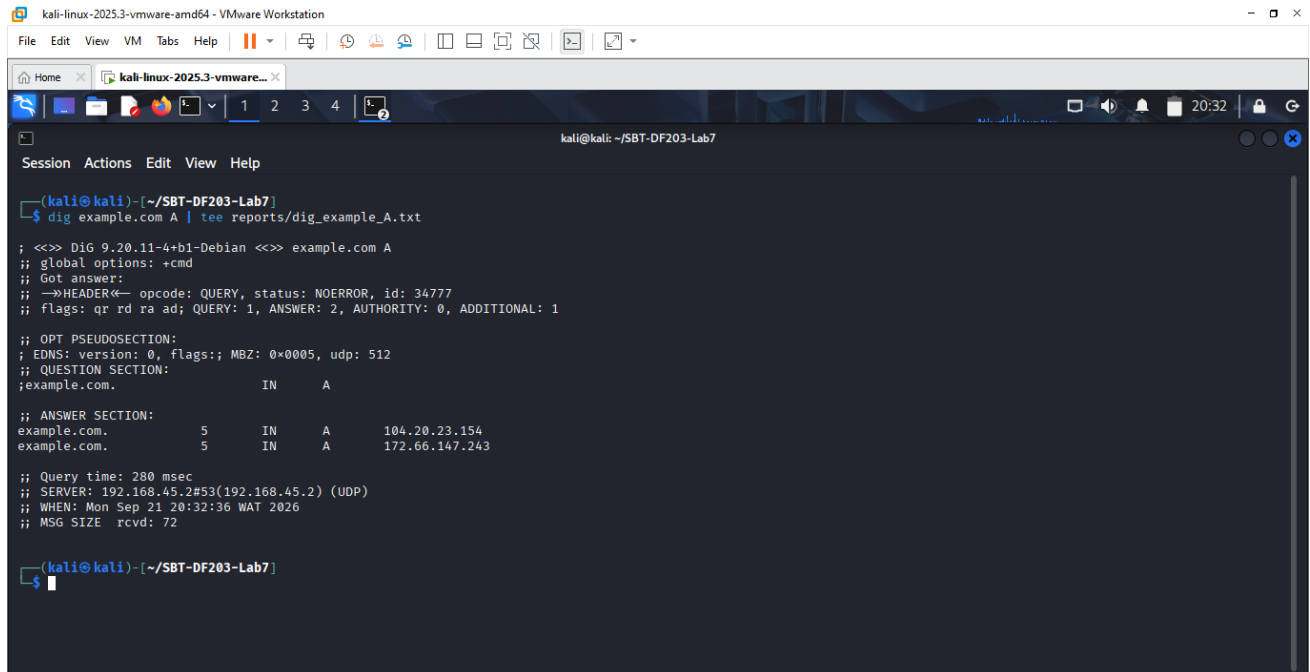
Explanation

The authorised dig_dns.pcap training capture was downloaded and copied into the working directory for analysis.

SHA-256 values were calculated for the original and working copies so their integrity could be compared.

Matching hashes showed that the working copy had not changed from the preserved source evidence.

Part A - Query DNS Records with dig



```
kali@kali: ~/SBT-DF203-Lab7
$ dig example.com A | tee reports/dig_example_A.txt

;<<>> DiG 9.20.11-4+b1-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 34777
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;example.com.                IN      A
;; ANSWER SECTION:
example.com.                  5       IN      A       104.20.23.154
example.com.                  5       IN      A       172.66.147.243

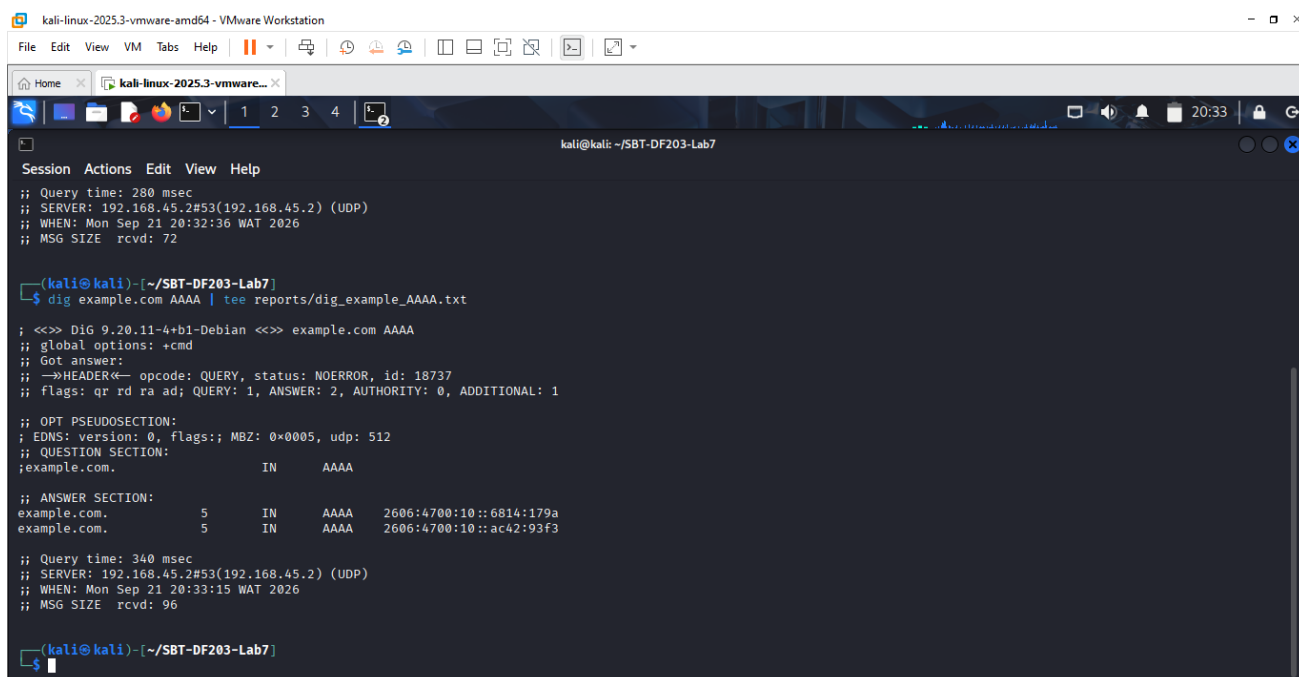
;; Query time: 280 msec
;; SERVER: 192.168.45.2#53(192.168.45.2) (UDP)
;; WHEN: Mon Sep 21 20:32:36 WAT 2026
;; MSG SIZE rcvd: 72

kali@kali: ~/SBT-DF203-Lab7
```

Fig 1.5 Dig A outputs.”.

Explanation

The A-record query for example.com returned IPv4 addresses and completed with a NOERROR DNS status. The output also displayed the resolver, answer count, flags, TTL and query time needed for the lab analysis. This demonstrated how an A record maps a domain name to IPv4 addresses



```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help

;; Query time: 280 msec
;; SERVER: 192.168.45.2#53(192.168.45.2) (UDP)
;; WHEN: Mon Sep 21 20:32:36 WAT 2026
;; MSG SIZE rcvd: 72

(kali@kali)~/SBT-DF203-Lab7
$ dig example.com AAAA | tee reports/dig_example_AAAA.txt

;<>> DiG 9.20.11-4+b1-Debian <>> example.com AAAA
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 18737
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;example.com. IN AAAA

;; ANSWER SECTION:
example.com. 5 IN AAAA 2606:4700:10::6814:179a
example.com. 5 IN AAAA 2606:4700:10::ac42:93f3

;; Query time: 340 msec
;; SERVER: 192.168.45.2#53(192.168.45.2) (UDP)
;; WHEN: Mon Sep 21 20:33:15 WAT 2026
;; MSG SIZE rcvd: 96

(kali@kali)~/SBT-DF203-Lab7
$
```

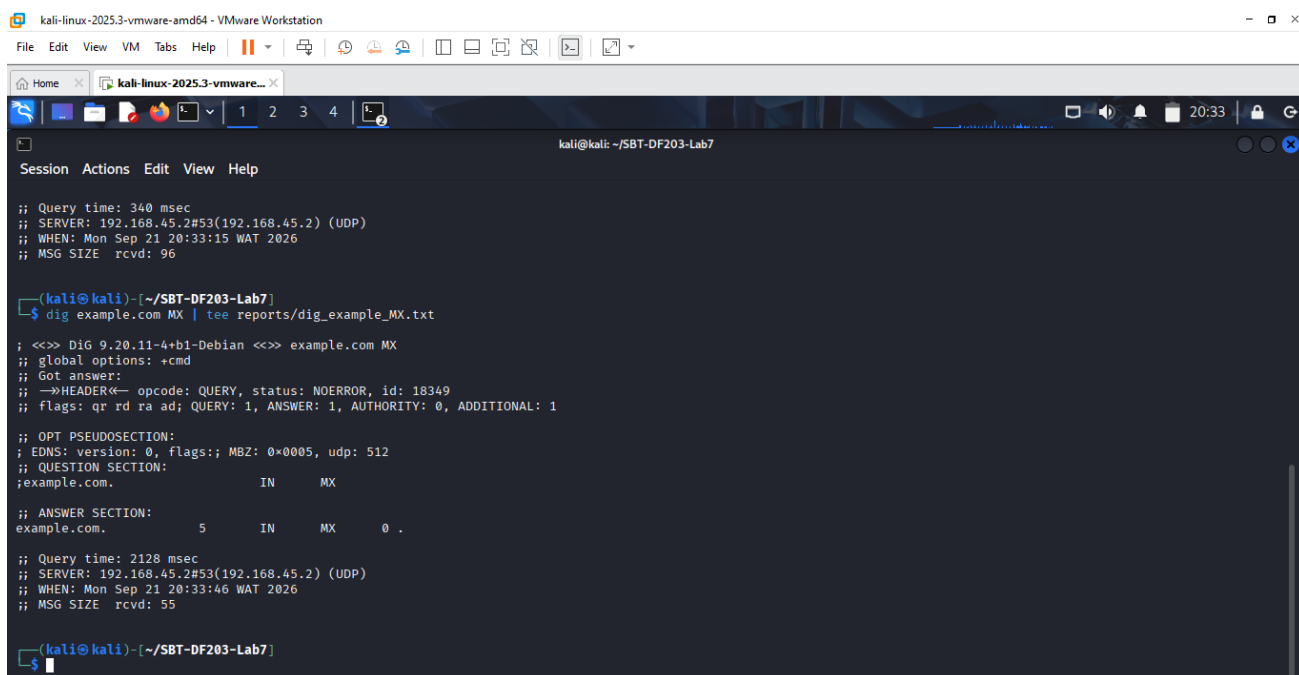
Fig 1.6 Dig AAAA outputs.”.

Explanation

The AAAA query returned IPv6 addresses for example.com.

The result showed that DNS can provide IPv6 information separately from IPv4 A records.

The response also included the status, resolver information, TTL and timing used to interpret the DNS reply.



```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help

;; Query time: 340 msec
;; SERVER: 192.168.45.2#53(192.168.45.2) (UDP)
;; WHEN: Mon Sep 21 20:33:15 WAT 2026
;; MSG SIZE rcvd: 96

(kali@kali)~/SBT-DF203-Lab7
$ dig example.com MX | tee reports/dig_example_MX.txt

;<>> DiG 9.20.11-4+b1-Debian <>> example.com MX
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 18349
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;example.com. IN MX

;; ANSWER SECTION:
example.com. 5 IN MX 0 .

;; Query time: 2128 msec
;; SERVER: 192.168.45.2#53(192.168.45.2) (UDP)
;; WHEN: Mon Sep 21 20:33:46 WAT 2026
;; MSG SIZE rcvd: 55

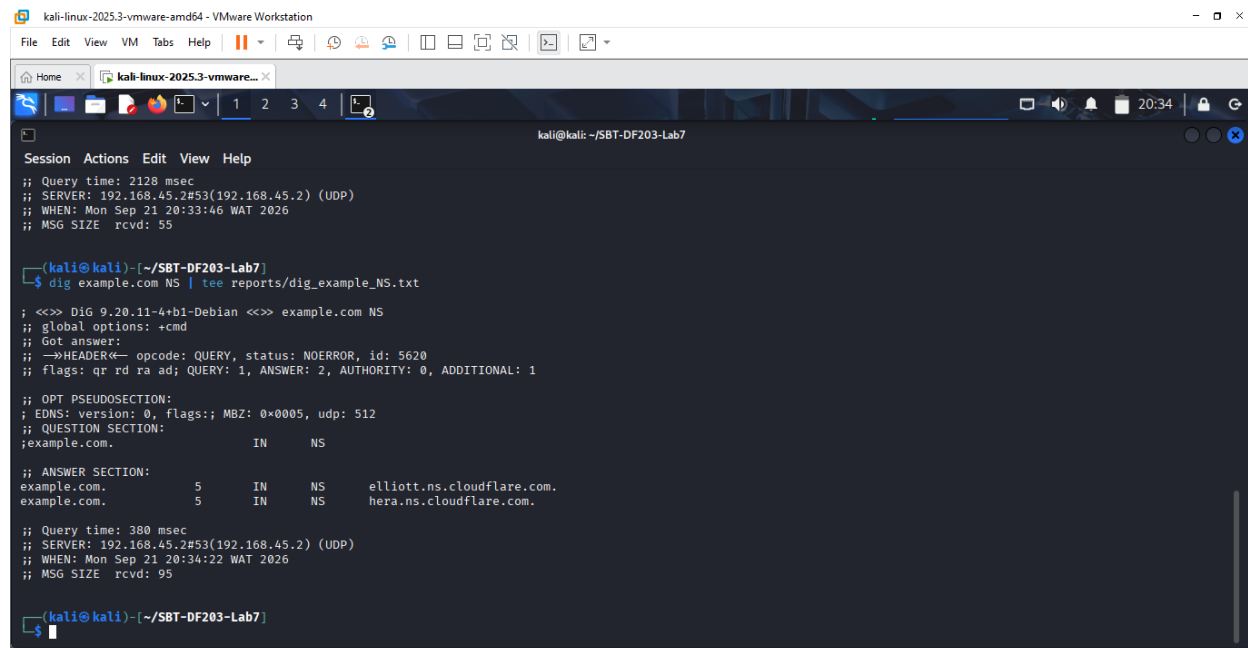
(kali@kali)~/SBT-DF203-Lab7
$
```

Fig 1.7 Dig MX outputs.”

Explanation

The MX query requested the mail-exchange information associated with example.com.

The response completed successfully and returned the MX information supplied by the DNS resolver. This illustrated that DNS contains service-related records in addition to ordinary IP-address mappings.



```
kali@kali: ~/SBT-DF203-Lab7
$ dig example.com NS | tee reports/dig_example_NS.txt

;<<>> DiG 9.20.11-4+b1-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;;->HEADER<- opcode: QUERY, status: NOERROR, id: 5620
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;example.com.                IN      NS

;; ANSWER SECTION:
example.com.                5       IN      NS      elliott.ns.cloudflare.com.
example.com.                5       IN      NS      hera.ns.cloudflare.com.

;; Query time: 380 msec
;; SERVER: 192.168.45.2#53(192.168.45.2) (UDP)
;; WHEN: Mon Sep 21 20:34:22 WAT 2026
;; MSG SIZE rcvd: 95

(kali@kali)~/SBT-DF203-Lab7
```

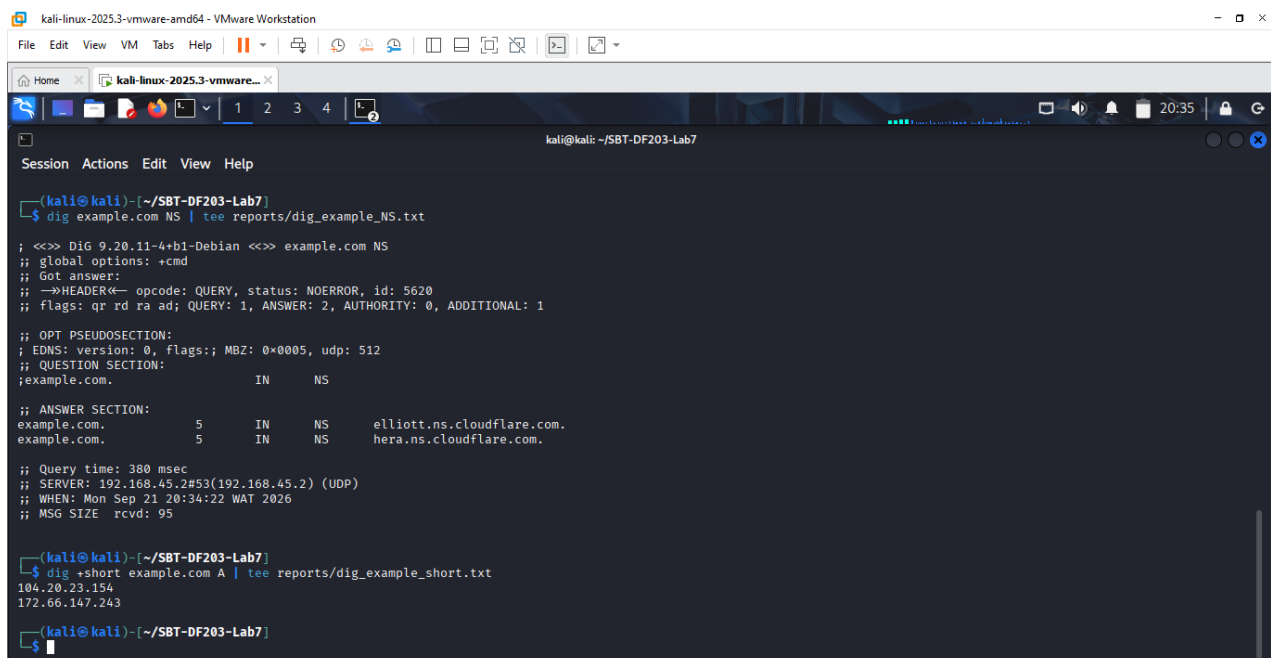
Fig 1.8 Dig NS outputs.”.

Explanation

The NS query identified the name servers associated with example.com.

The response returned elliott.ns.cloudflare.com and hera.ns.cloudflare.com.

These records show which DNS servers are responsible for authoritative information about the domain.



```
kali@kali: ~/SBT-DF203-Lab7
$ dig +short example.com A | tee reports/dig_example_short.txt

104.20.23.154
172.66.147.243

(kali@kali)~/SBT-DF203-Lab7
```

Fig 1.9 Dig example_short.txt outputs.”.

Explanation

The short-format query returned only the IPv4 answers instead of the full DNS response details.

The addresses returned were 104.20.23.154 and 172.66.147.243.

This demonstrated how dig +short can be useful when only the resolved values are required.

What is an A record?

An A record maps a hostname or domain to an IPv4 address.

In this lab:

example.com

returned:

104.20.23.154

172.66.147.243

What is an AAAA record?

An AAAA record maps a hostname or domain to an IPv6 address.

The AAAA query in this practical returned IPv6 addresses for example.com, showing the IPv6 equivalent of an A-record lookup.

What is an MX record?

An MX record identifies the mail-exchange information associated with a domain.

It is used to determine where email for that domain should be delivered.

The lab included:

dig example.com MX

as one of the required DNS queries.

What is an NS record?

An NS record identifies the authoritative name servers associated with a domain.

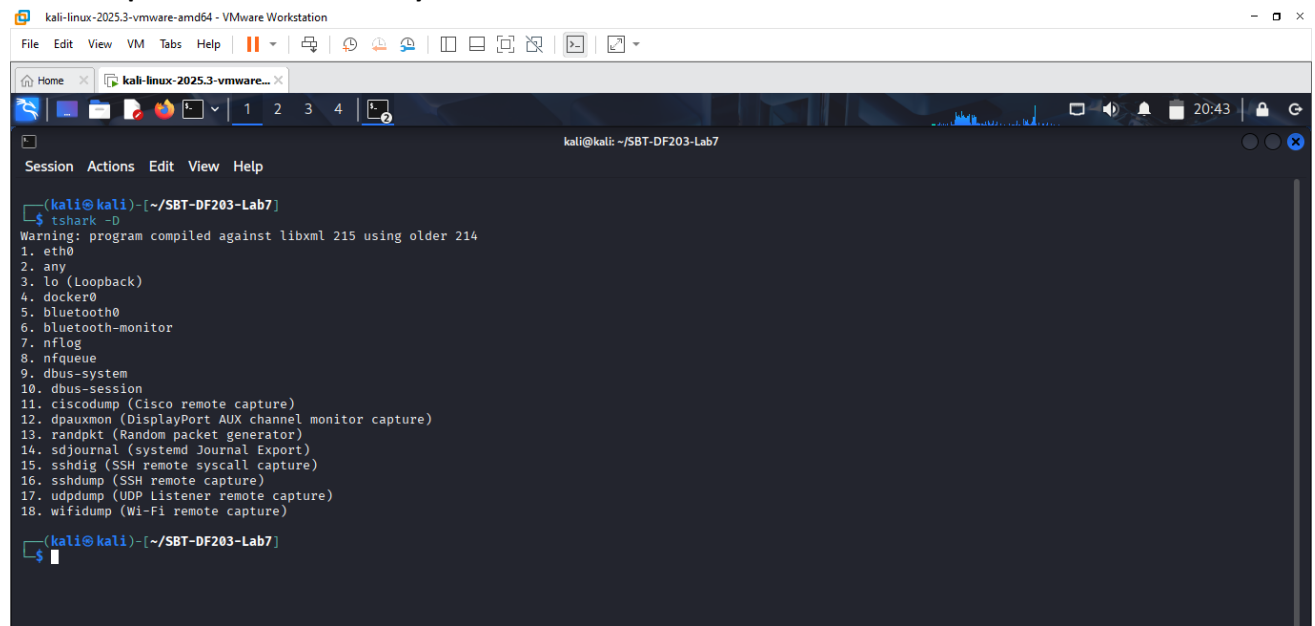
For example.com, this practical returned:

elliott.ns.cloudflare.com

hera.ns.cloudflare.com

These servers hold or provide authoritative DNS information for the domain.

Part B - Capture a Fresh DNS Query

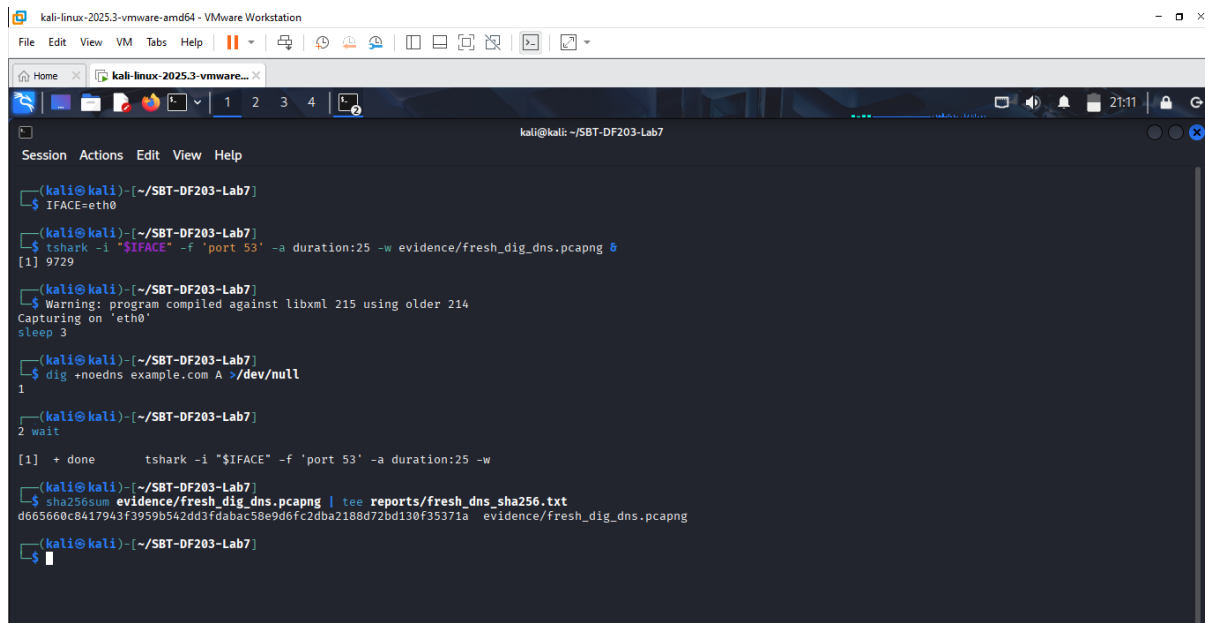


```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
(kali@kali)~/SBT-DF203-Lab7
$ tshark -D
Warning: program compiled against libxml 215 using older 214
1. eth0
2. any
3. lo (Loopback)
4. docker0
5. bluetooth0
6. bluetooth-monitor
7. nflog
8. nfqueue
9. dbus-system
10. dbus-session
11. ciscodump (Cisco remote capture)
12. dpauemon (DisplayPort AUX channel monitor capture)
13. randpkt (Random packet generator)
14. sdjournal (systemd Journal Export)
15. sshdig (SSH remote syscall capture)
16. sshdump (SSH remote capture)
17. udpdump (UDP Listener remote capture)
18. wifidump (Wi-Fi remote capture)
(kali@kali)~/SBT-DF203-Lab7
$
```

Fig 2.0 Confirming your active Ethernet.

Explanation

TShark listed the available capture interfaces and confirmed that eth0 was available for packet capture. Identifying the correct interface was necessary before starting the fresh DNS recording. This avoided capturing from an unrelated interface such as loopback or a virtual monitoring source.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
(kali@kali)~/SBT-DF203-Lab7
$ IFACE=eth0
(kali@kali)~/SBT-DF203-Lab7
$ tshark -i "$IFACE" -f 'port 53' -a duration:25 -w evidence/fresh_dig_dns.pcapng 0
[1] 9729
(kali@kali)~/SBT-DF203-Lab7
$ Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
sleep 3
(kali@kali)~/SBT-DF203-Lab7
$ dig +noedns example.com A >/dev/null
1
(kali@kali)~/SBT-DF203-Lab7
2 wait
[1] + done tshark -i "$IFACE" -f 'port 53' -a duration:25 -w
(kali@kali)~/SBT-DF203-Lab7
$ sha256sum evidence/fresh_dig_dns.pcapng | tee reports/fresh_dns_sha256.txt
d665660c8417943f3959b542dd3fdabac58e9d6fc2dba2188d72bd130f35371a evidence/fresh_dig_dns.pcapng
(kali@kali)~/SBT-DF203-Lab7
$
```

Fig 2.1 Fresh DNS Capture Hash.

Explanation

A fresh DNS query was captured on eth0 and saved as fresh_dig_dns.pcapng. The capture was then hashed with SHA-256 so the evidence could be preserved and verified. This created an integrity record for the newly generated DNS evidence before detailed analysis.

Part C - Extract DNS Query and Response Fields

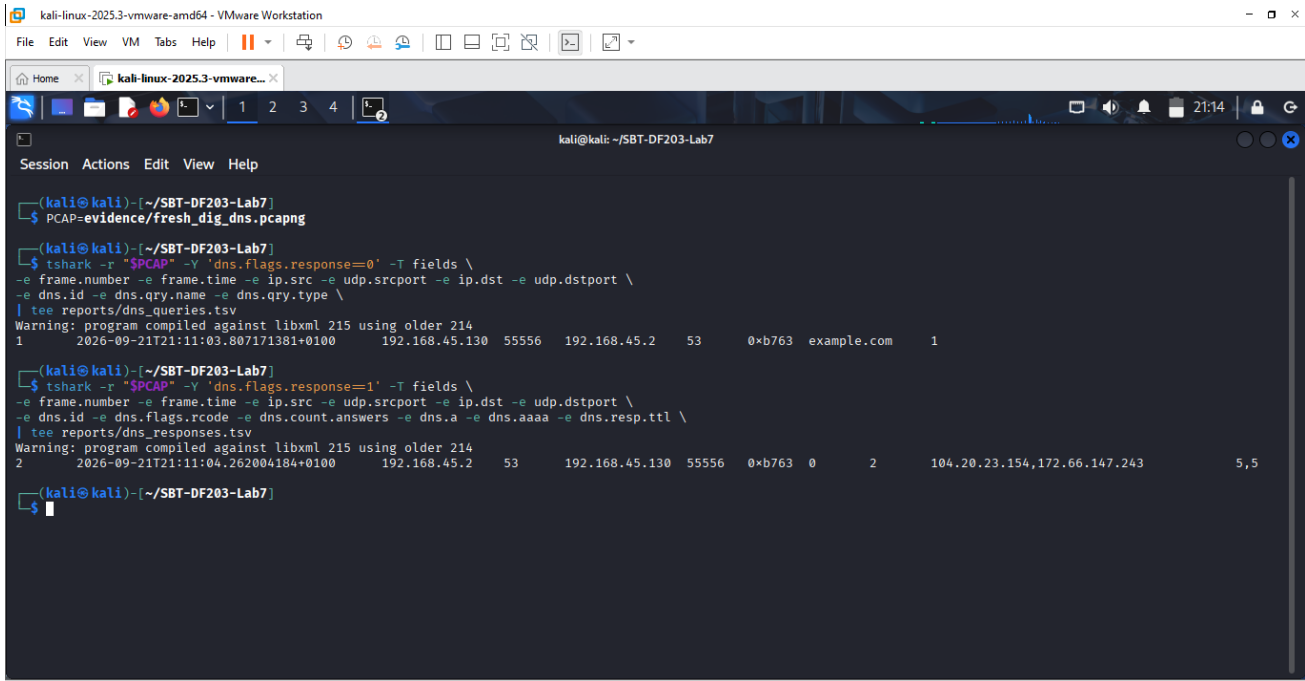


Fig 2.2 Extracting DNS Query and Response Fields.

Explanation

TShark extracted the DNS query and matching response from the fresh capture. The query used transaction ID 0xb763, client port 55556, resolver 192.168.45.2, and requested an A record for example.com.

The matching response returned two IPv4 addresses with response code 0, confirming a successful DNS transaction.

Part D - Match Queries to Responses

Transaction ID	Query Name	Type	Client/Resolver	Response Code	Answer(s)	TTL	Time Delta
0xb763	example.com	A	192.168.45.130:555	0	104.20.23.154	5 s	454.833 ms
			56 ↔	(NOERROR)	172.66.147.243		
			192.168.45.2:53				

The table brings the query and response information together using the transaction ID and client/resolver endpoints.

It shows that example.com resolved to 104.20.23.154 and 172.66.147.243 with a TTL of 5 seconds.

The calculated response delay was approximately 454.833 ms, showing how quickly the DNS reply followed the request.

What is a CNAME record?

A CNAME, or canonical-name record, maps one hostname to another hostname rather than directly to an IP address.

The optional SMTP evidence showed:

mail.patriots.in

↓

patriots.in

↓

74.53.140.153

The first relationship was a CNAME, while the final result was an A record.

What is TTL?

TTL means Time To Live.

It tells a resolver how long a DNS record may be cached before it should be considered expired and queried again.

For the fresh example.com A-record response, the TTL was:

5 seconds

TTL is a cache lifetime, not the date when the DNS record was created. The manual explicitly requires that distinction.

What does NOERROR mean?

NOERROR means the DNS request was processed successfully without a DNS protocol error.

The equivalent response code is:

0

The controlled example.com query returned:

0 (NOERROR)

Part E - Analyze DNS Generated by a Browser

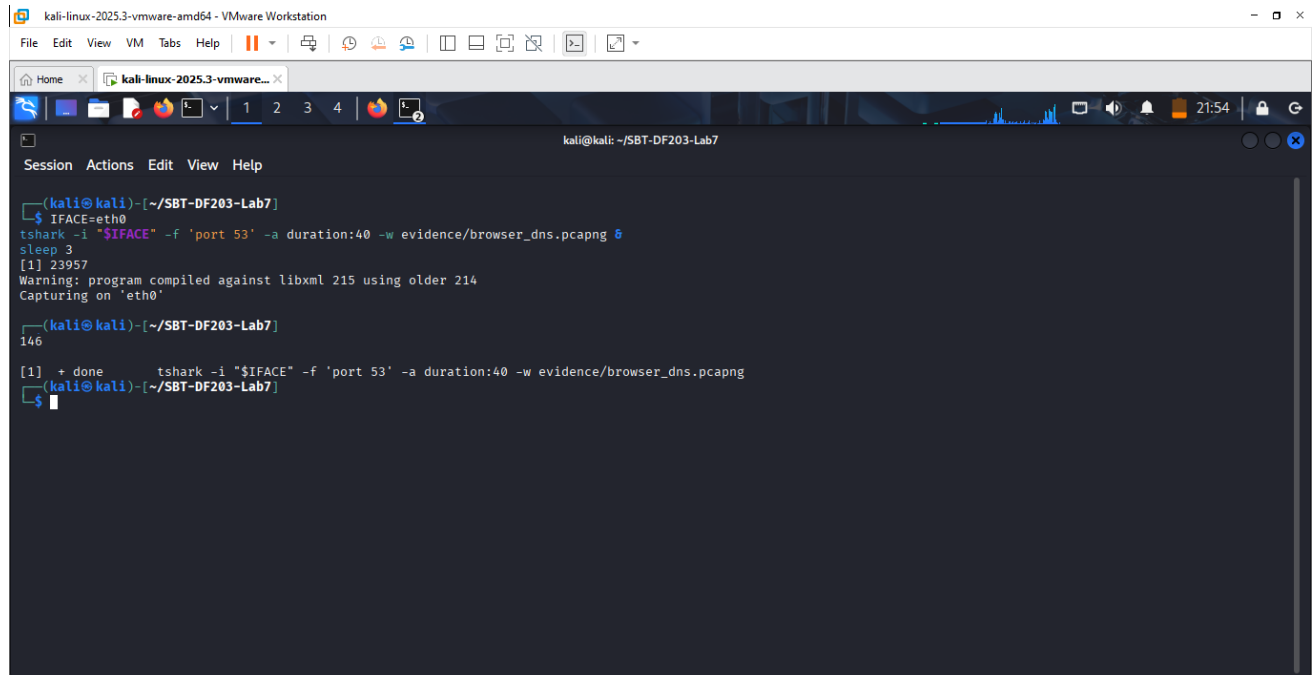


Fig 2.3 Analyzing DNS Generated by a Browser.

Explanation

Shark was started to collect DNS traffic while browser activity was generated.

The capture provided a broader example of DNS behaviour than the single controlled dig query.

Browser-generated traffic was later examined to identify the different domains automatically requested by the browser.

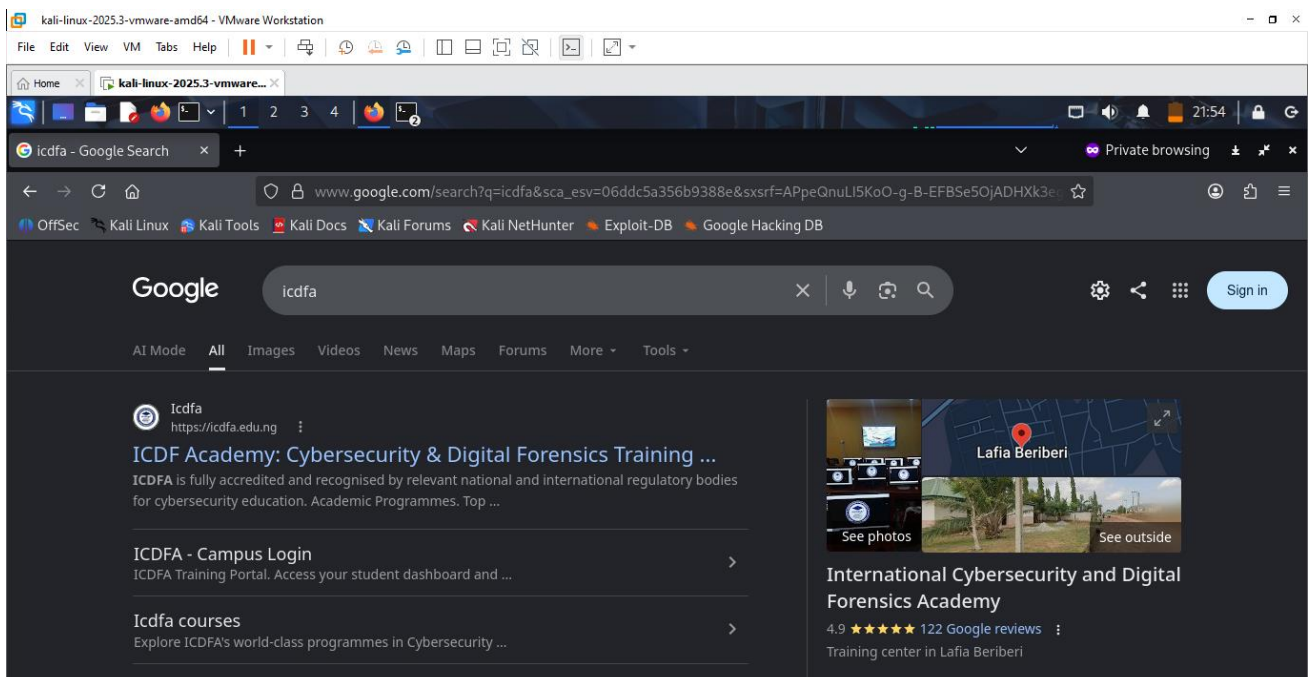


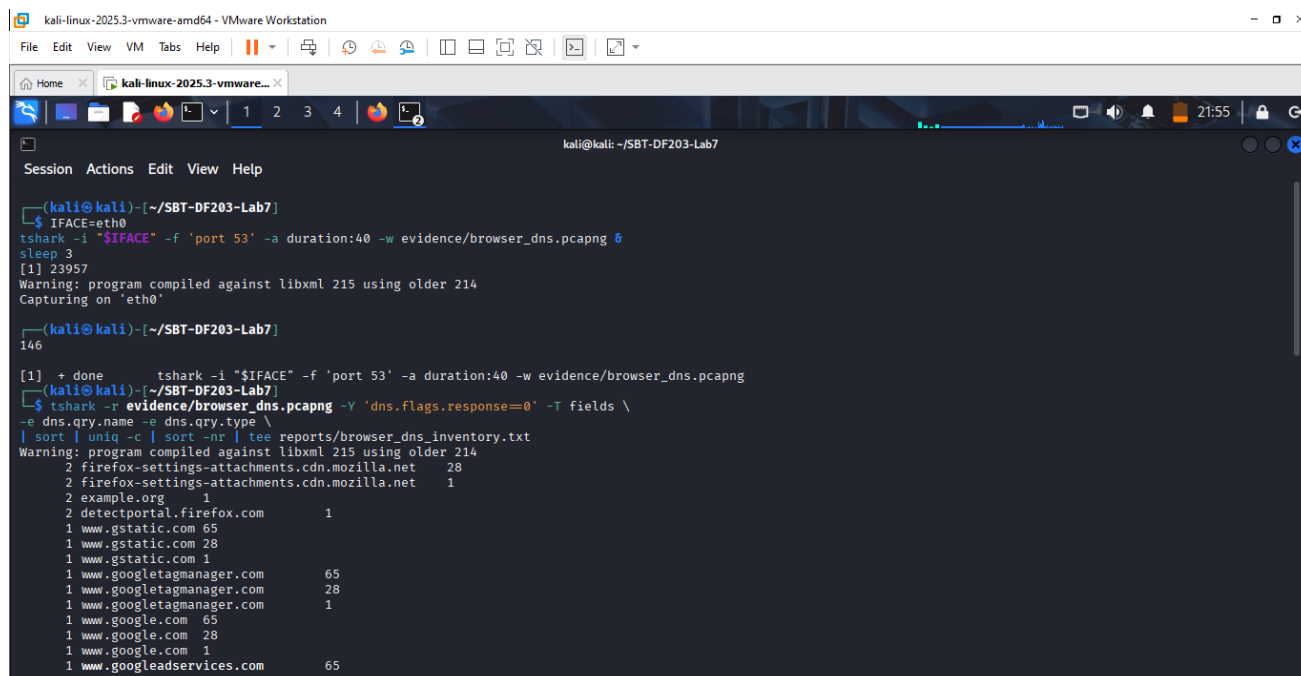
Fig 2.4 Analyzing DNS Generated by a Browser page 1.

Explanation

Firefox Private Browsing was used to access Google and perform a search during the DNS capture.

This generated realistic browser-related DNS traffic rather than relying only on manually issued queries.

The browsing activity provided the source for the later DNS inventory and connection-correlation analysis.



```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help

(kali@kali)~[~/SBT-DF203-Lab7]
$ IFACE=eth0
tshark -i "$IFACE" -f 'port 53' -a duration:40 -w evidence/browser_dns.pcapng &
sleep 3
[1] 23957
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'

(kali@kali)~[~/SBT-DF203-Lab7]
146
[1] + done tshark -i "$IFACE" -f 'port 53' -a duration:40 -w evidence/browser_dns.pcapng
(kali@kali)~[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields \
-e dns.qry.name -e dns.qry.type \
| sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
Warning: program compiled against libxml 215 using older 214
 2 firefox-settings-attachments.cdn.mozilla.net 28
 2 firefox-settings-attachments.cdn.mozilla.net 1
 2 example.org 1
 2 detectportal.firefox.com 1
 1 www.gstatic.com 65
 1 www.gstatic.com 28
 1 www.gstatic.com 1
 1 www.googletagmanager.com 65
 1 www.googletagmanager.com 28
 1 www.googletagmanager.com 1
 1 www.google.com 65
 1 www.google.com 28
 1 www.google.com 1
 1 www.googleadservices.com 65
```

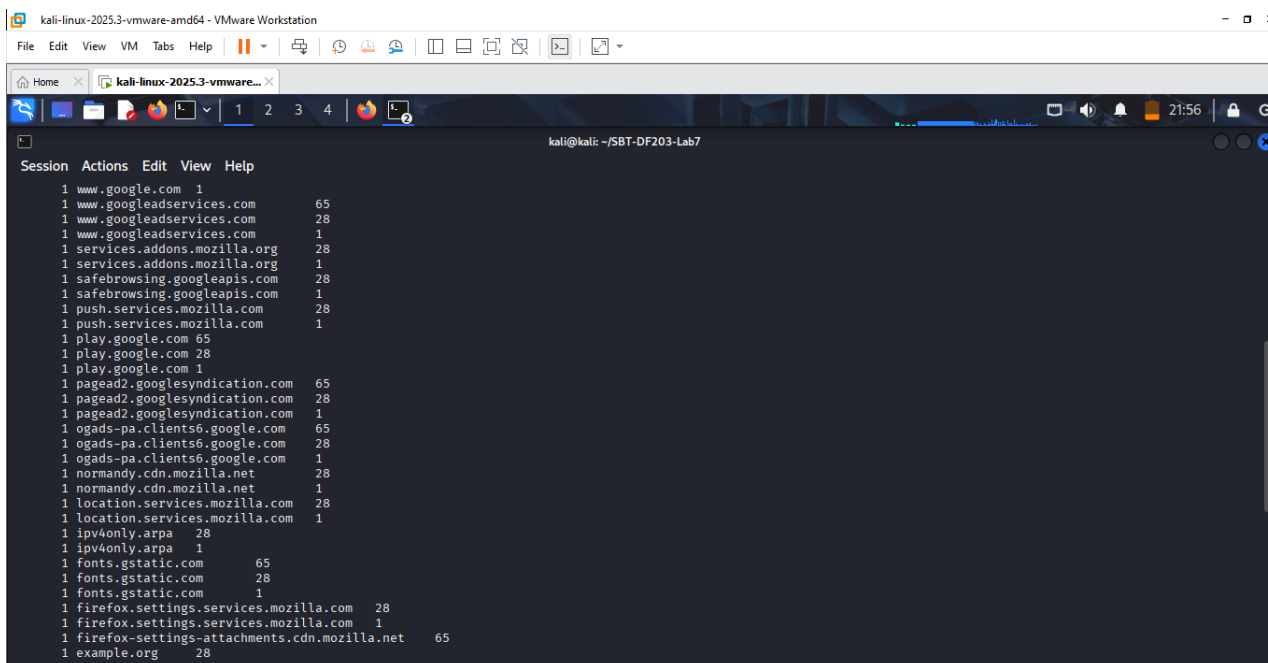
Fig 2.5 Analyzing DNS Generated by a Browser page 2.

Explanation

The DNS inventory shows several domains requested automatically during the Google browsing session.

Entries included Google services, Mozilla services and supporting web resources.

The output demonstrates that loading one web page can cause many DNS lookups in the background.



```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help

 1 www.google.com 1
 1 www.googleadservices.com 65
 1 www.googleadservices.com 28
 1 www.googleadservices.com 1
 1 services.addons.mozilla.org 28
 1 services.addons.mozilla.org 1
 1 safebrowsing.googleapis.com 28
 1 safebrowsing.googleapis.com 1
 1 push.services.mozilla.com 28
 1 push.services.mozilla.com 1
 1 play.google.com 65
 1 play.google.com 28
 1 play.google.com 1
 1 pagead2.googlesyndication.com 65
 1 pagead2.googlesyndication.com 28
 1 pagead2.googlesyndication.com 1
 1 ogads-pa.clients6.google.com 65
 1 ogads-pa.clients6.google.com 28
 1 ogads-pa.clients6.google.com 1
 1 normandy.cdn.mozilla.net 28
 1 normandy.cdn.mozilla.net 1
 1 location.services.mozilla.com 28
 1 location.services.mozilla.com 1
 1 ipv4only.arpa 28
 1 ipv4only.arpa 1
 1 fonts.gstatic.com 65
 1 fonts.gstatic.com 28
 1 fonts.gstatic.com 1
 1 firefox.settings.services.mozilla.com 28
 1 firefox.settings.services.mozilla.com 1
 1 firefox-settings-attachments.cdn.mozilla.net 65
 1 example.org 28
```

Fig 2.6 Analyzing DNS Generated by a Browser page 3.

Explanation

This section continues the inventory of browser-generated DNS queries.

It shows domains related to fonts, advertising, Google services, Firefox components and other web resources.

The evidence confirms that browser DNS traffic is considerably more varied than a single command-line lookup.



```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
1 firefox.settings.services.mozilla.com 28
1 firefox.settings.services.mozilla.com 1
1 firefox.settings.attachments.cdn.mozilla.net 65
1 example.org 28
1 ep2.adtrafficquality.google 65
1 ep2.adtrafficquality.google 28
1 ep2.adtrafficquality.google 1
1 ep1.adtrafficquality.google 65
1 ep1.adtrafficquality.google 28
1 ep1.adtrafficquality.google 1
1 encrypted-tbn0.gstatic.com 65
1 encrypted-tbn0.gstatic.com 28
1 encrypted-tbn0.gstatic.com 1
1 detectportal.firefox.com 28
1 csp.withgoogle.com 65
1 csp.withgoogle.com 28
1 csp.withgoogle.com 1
1 content-signature-2.cdn.mozilla.net 28
1 content-signature-2.cdn.mozilla.net 1
1 consent.google.com 65
1 consent.google.com 28
1 consent.google.com 1
1 cloudflare-dns.com 65
1 classify-client.services.mozilla.com 28
1 classify-client.services.mozilla.com 1
1 aus5.mozilla.org 28
1 aus5.mozilla.org 1
1 ads.mozilla.org 28
1 ads.mozilla.org 1
```

Fig 2.7 Analyzing DNS Generated by a Browser page 4.

Explanation

The final section of the browser inventory shows more Google and Mozilla-related requests produced during the browsing session.

Multiple DNS query types were present, showing that the browser did not rely on only one record type. This behaviour is normal because modern web pages load content and services from several different domains.

Part F - Correlate DNS with Subsequent Connections

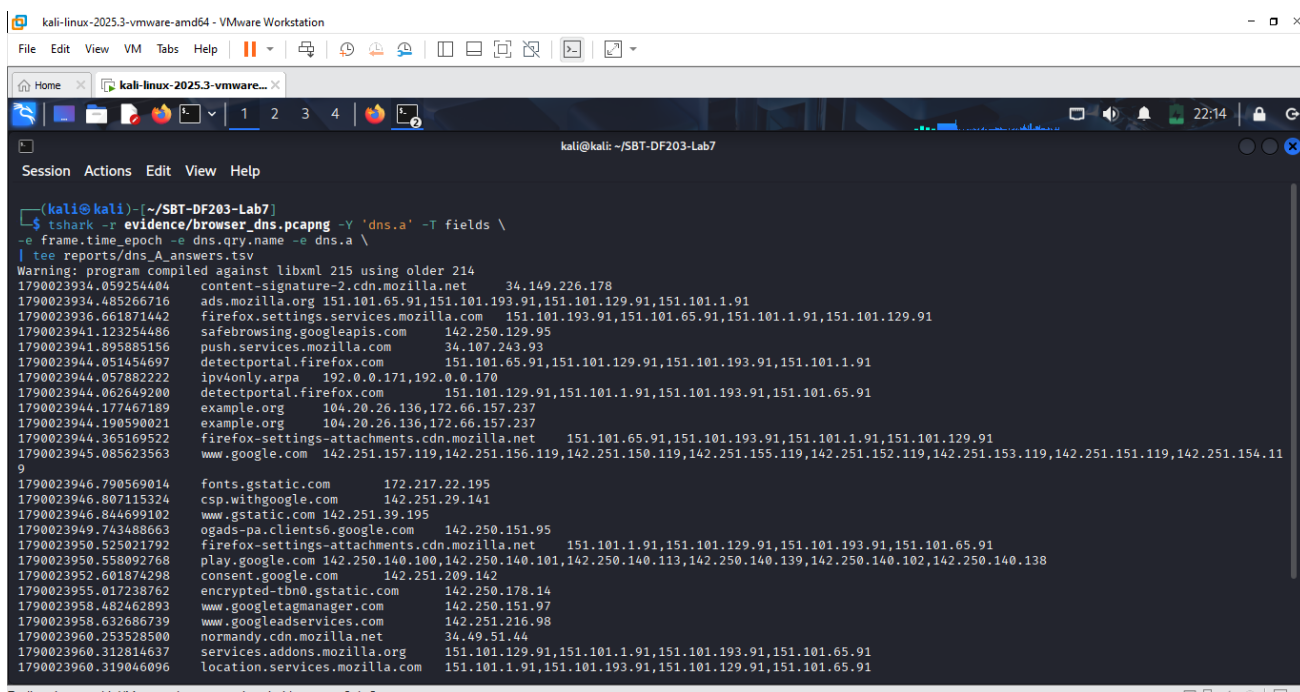


Fig 2.8 Correlate DNS with Subsequent Connections.

Explanation

DNS A-record answers were extracted from the browser capture to identify domain-to-IP mappings. The output provided timestamps, query names and returned IPv4 addresses for later comparison. These values formed the DNS side of the DNS-to-connection correlation exercise.

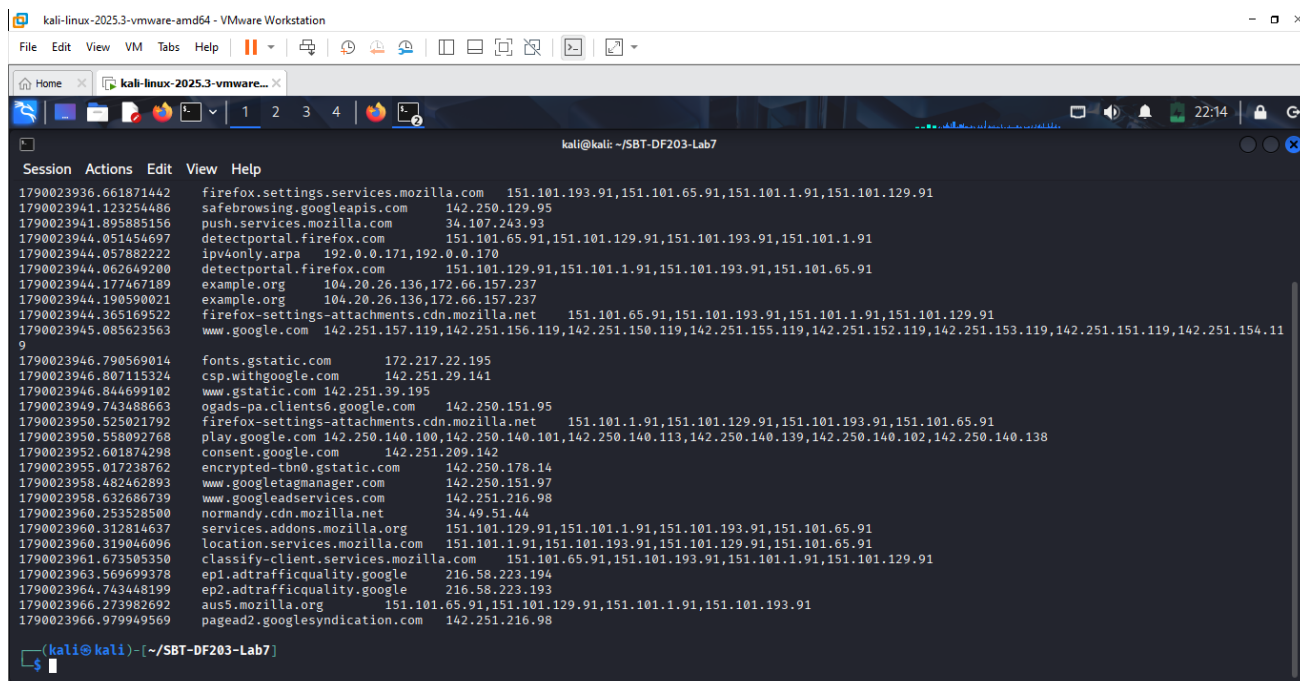
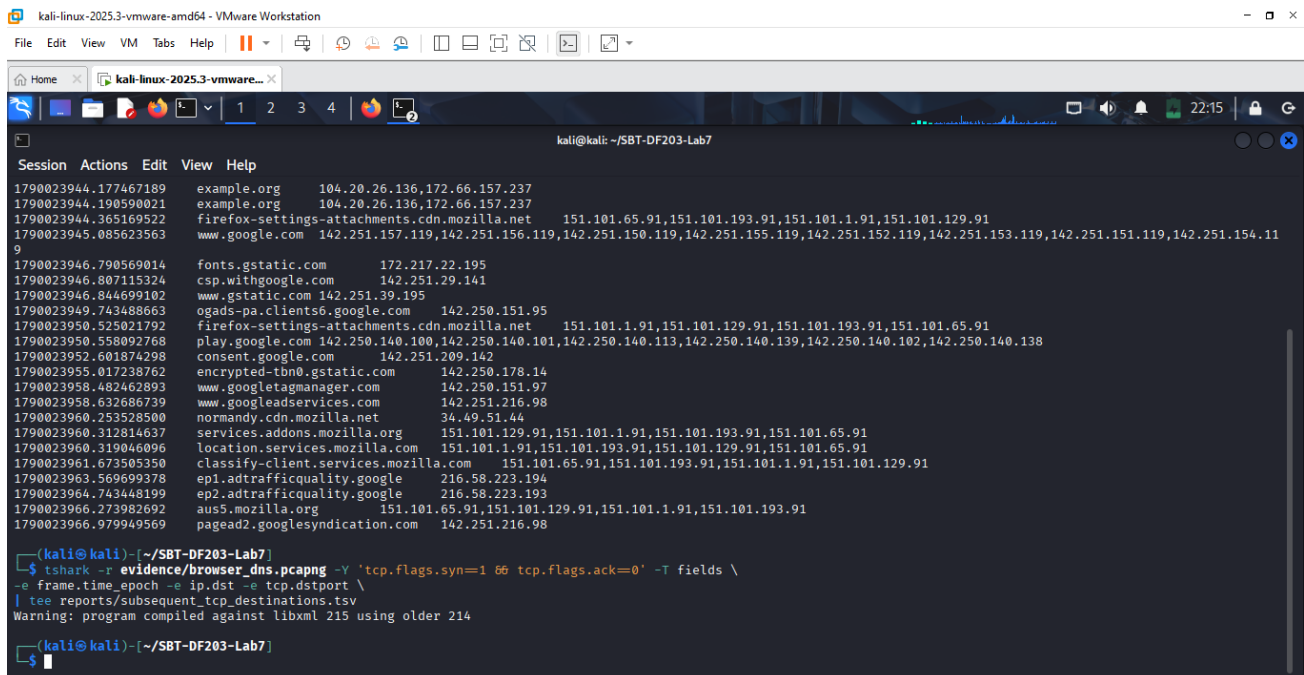


Fig 2.9 Correlate DNS with Subsequent Connections page 1.

Explanation

This screenshot continues the list of returned A-record addresses from browser DNS activity.

Several domains returned multiple IP addresses, which is normal for distributed web services. These answers were compared with later network destinations to identify whether the browser connected to a resolved host.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
1790023944.177467189 example.org 104.20.26.136,172.66.157.237
1790023944.198590021 example.org 104.20.26.136,172.66.157.237
1790023944.365169522 firefox-settings-attachments.cdn.mozilla.net 151.101.65.91,151.101.193.91,151.101.1.91,151.101.129.91
1790023945.085623563 www.google.com 142.251.157.119,142.251.156.119,142.251.150.119,142.251.155.119,142.251.152.119,142.251.153.119,142.251.151.119,142.251.154.11
9
1790023946.790569014 fonts.gstatic.com 172.217.22.195
1790023946.807115324 csp.withgoogle.com 142.251.29.141
1790023946.844699102 www.gstatic.com 142.251.39.195
1790023949.743488663 ogads-pa.clients6.google.com 142.250.151.95
1790023950.525021792 firefox-settings-attachments.cdn.mozilla.net 151.101.1.91,151.101.129.91,151.101.193.91,151.101.65.91
1790023950.558092768 play.google.com 142.250.140.100,142.250.140.101,142.250.140.113,142.250.140.139,142.250.140.102,142.250.140.138
1790023952.601874298 consent.google.com 142.251.209.142
1790023955.017238762 encrypted-tbn0.gstatic.com 142.250.178.14
1790023958.482462893 www.googletagmanager.com 142.250.151.97
1790023958.632686739 www.googleadservices.com 142.251.216.98
1790023960.253528500 normandy.cdn.mozilla.net 34.49.51.44
1790023960.312814637 services.addons.mozilla.org 151.101.129.91,151.101.1.91,151.101.193.91,151.101.65.91
1790023960.319046096 location.services.mozilla.com 151.101.1.91,151.101.193.91,151.101.129.91,151.101.65.91
1790023961.673505350 classify-client.services.mozilla.com 151.101.65.91,151.101.193.91,151.101.1.91,151.101.129.91
1790023963.569699378 epl.adtrafficquality.google 216.58.223.104
1790023964.742448199 ep2.adtrafficquality.google 216.58.223.103
1790023966.273982692 aus5.mozilla.org 151.101.65.91,151.101.129.91,151.101.1.91,151.101.193.91
1790023966.979949569 pagead2.googleadsyndication.com 142.251.216.98
(kali@kali)~[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.time_epoch -e ip.dst -e tcp.dstport \
| tee reports/subsequent_tcp_destinations.tsv
Warning: program compiled against libxml 215 using older 214
(kali@kali)~[~/SBT-DF203-Lab7]
$
```

Fig 3.0 Correlate DNS with Subsequent Connections page 2.

Explanation

The first browser capture contained DNS-only traffic because it had been recorded using a port-53 capture filter.

As a result, the later TCP SYN filter produced no destination entries from that file.

A separate correlation capture was therefore created without the DNS-only capture restriction so both DNS and later TCP traffic could be preserved.

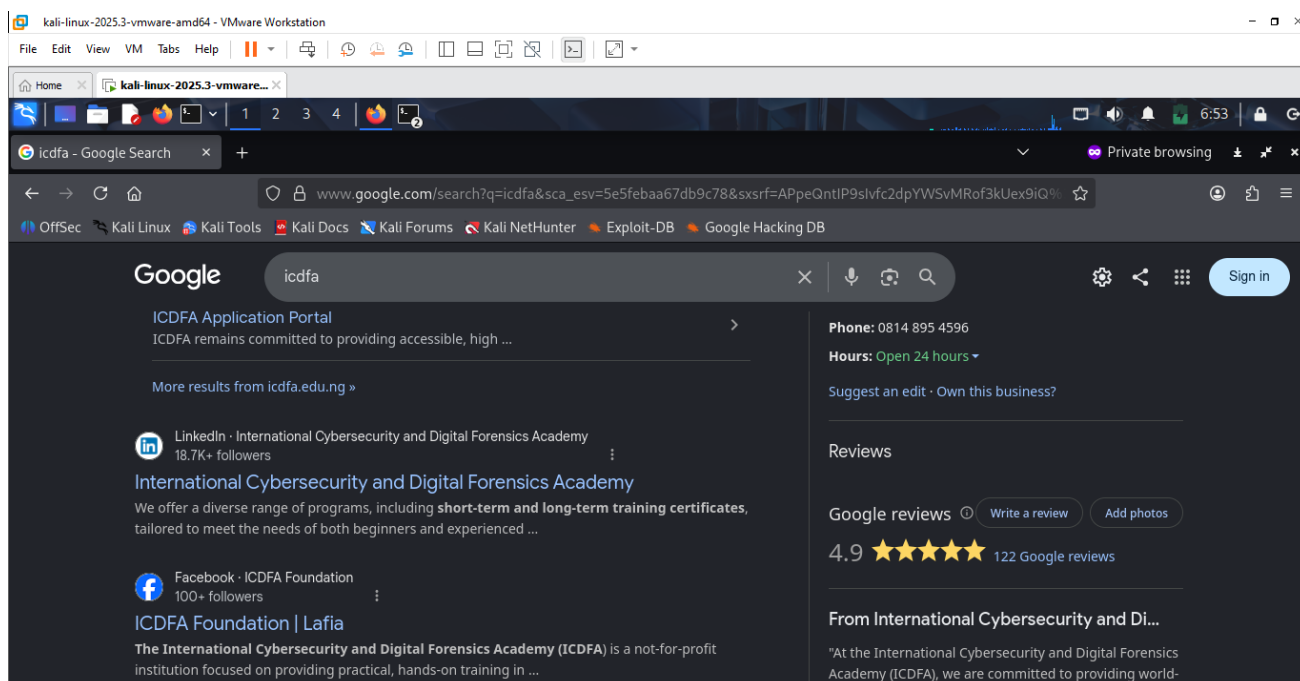


Fig 3.0.1 Correlate DNS with Subsequent Connections web connection.

Explanation

The browser was used again during the broader correlation capture so DNS resolution and subsequent connection activity could be recorded together.

Google browsing provided realistic HTTPS connection traffic for comparison with the DNS results.

This second capture allowed a proper relationship between a resolved IP and a following TCP connection to be demonstrated.

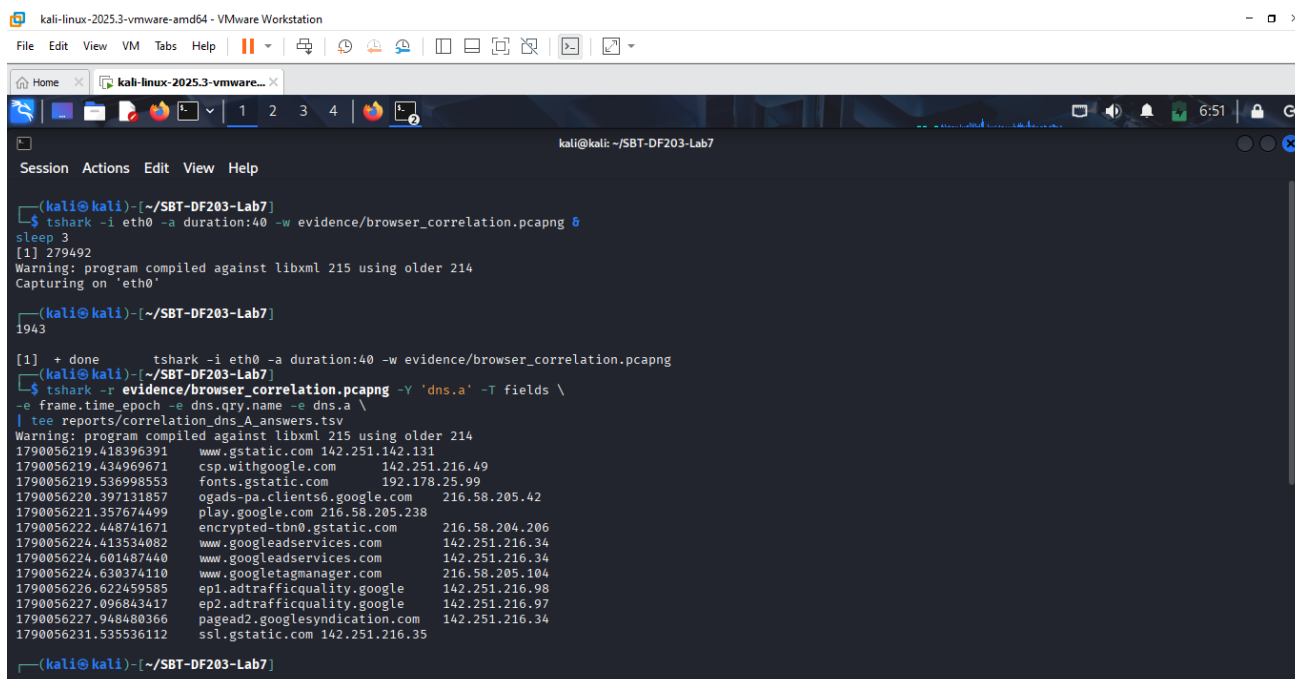


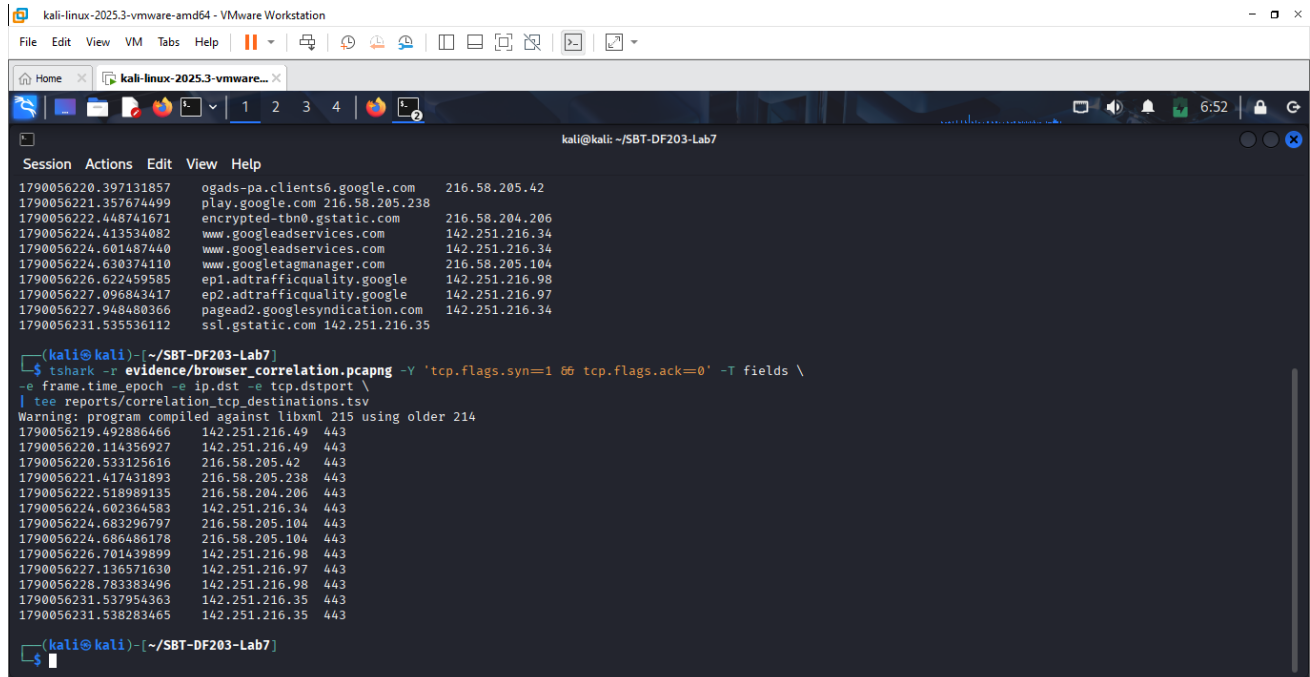
Fig 3.1 TCP Destination Correlation.

Explanation

DNS answers were extracted from the new correlation capture and listed alongside their timestamps.

The results included ssl.gstatic.com resolving to 142.251.216.35.

This IP was later found among the TCP destinations, making it suitable for the final correlation finding.



```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
1790056220.397131857 ogads-pa.clients6.google.com 216.58.205.42
1790056221.357674499 play.google.com 216.58.205.238
1790056222.448741671 encrypted-tbn0.gstatic.com 216.58.204.206
1790056224.413534082 www.googleadservices.com 142.251.216.34
1790056224.601487440 www.googleadservices.com 142.251.216.34
1790056224.630374110 www.googletagmanager.com 216.58.205.104
1790056226.622459585 ep1.adtrafficquality.google 142.251.216.98
1790056227.096843417 ep2.adtrafficquality.google 142.251.216.97
1790056227.948480366 pagead2.googleadsyndication.com 142.251.216.34
1790056231.535536112 ssl.gstatic.com 142.251.216.35

(kali@kali)~-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_correlation.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.time_epoch -e ip.dst -e tcp.dstport \
| tee reports/correlation_tcp_destinations.tsv
Warning: program compiled against libxml 215 using older 214
1790056219.492886466 142.251.216.49 443
1790056220.114356927 142.251.216.49 443
1790056220.533125616 216.58.205.42 443
1790056221.417431893 216.58.205.238 443
1790056222.518989135 216.58.204.206 443
1790056224.602364583 142.251.216.34 443
1790056224.683296797 216.58.205.104 443
1790056224.686486178 216.58.205.104 443
1790056226.701439899 142.251.216.98 443
1790056227.136571630 142.251.216.97 443
1790056228.783383496 142.251.216.98 443
1790056231.537954363 142.251.216.35 443
1790056231.538283465 142.251.216.35 443

(kali@kali)~-[~/SBT-DF203-Lab7]
```

Fig 3.2 TCP Destination Correlation page 1.

The TCP SYN analysis showed later connections to several IP addresses on port 443.

One of these destinations was 142.251.216.35, which had already appeared as the DNS answer for ssl.gstatic.com.

The connection followed the DNS response by approximately 2.418 ms, providing direct DNS-to-network correlation.

Explanation

The browser correlation capture showed a clear relationship between DNS resolution and subsequent network activity. The domain ssl.gstatic.com resolved to 142.251.216.35, and shortly afterward the client initiated a TCP connection to the same IP address on port 443. The time difference was approximately 2.418 milliseconds, providing direct evidence that the DNS response was followed by a connection to the resolved host.

Part G - DNS Analysis Within SMTP Evidence

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab7

(kali@kali)~/SBT-DF203-Lab7
$ tshark -r ../SBT-DF203-Lab4/working/smtp_working.pcap -Y 'dns' -T fields \
-e frame.number -e frame.time -e dns.qry.name -e dns.qry.type -e dns.a -e dns.resp.name -e dns.resp.ttl \
| tee reports/smtp_dns_correlation.tsv
Warning: program compiled against libxml 215 using older 214
1      2009-10-05T07:06:07.492060000+0100      mail.patriots.in      1
2      2009-10-05T07:06:07.526085000+0100      mail.patriots.in      1      74.53.140.153      mail.patriots.in,patriots.in,patriots.in,patriots.in      10827,10828,82
828,82828

(kali@kali)~/SBT-DF203-Lab7
$ tshark -r ../SBT-DF203-Lab4/working/smtp_working.pcap -Y 'dns.flags.response==1' -V \
| grep -E 'Name:|Type:|Time to live:|Address:'
Warning: program compiled against libxml 215 using older 214
Type: IPv4 (0x0800)
Source Address: 10.10.1.1
Destination Address: 10.10.1.4
Name: mail.patriots.in
Type: A (1) (Host Address)
Name: mail.patriots.in
Type: CNAME (5) (Canonical NAME for an alias)
Time to live: 10827 (3 hours, 27 seconds)
Name: patriots.in
Type: A (1) (Host Address)
Time to live: 10828 (3 hours, 28 seconds)
Address: 74.53.140.153
Name: patriots.in
Type: NS (2) (authoritative Name Server)
Time to live: 82828 (23 hours, 28 seconds)
Name: patriots.in
Type: NS (2) (authoritative Name Server)
Time to live: 82828 (23 hours, 28 seconds)

(kali@kali)~/SBT-DF203-Lab7
```

Fig 3.3 SMTP DNS Correlation.

Explanation

The optional Lab 4 SMTP evidence contained a DNS query for mail.patriots.in followed by its matching response.

The query used an A record and ultimately resolved to IPv4 address 74.53.140.153.

The response also showed the CNAME relationship with patriots.in, providing a useful example of DNS resolution within email-related traffic.

Field	Finding
Query frame	1
Response frame	2
Query name	mail.patriots.in
Query type	1 = A record
Answer IPv4	74.53.140.153
Response status	DNS response present
Resolver	10.10.1.1
Client	10.10.1.4

Field	Finding
Configured resolver IP	192.168.45.2

Field	Finding
Client source port	55556
Resolver destination port	53/UDP
Transaction ID	0xb763
Query name and type	example.com — Type A (IPv4 host address)
Response code	0 (NOERROR)
Answer IP(s)	104.20.23.154, 172.66.147.243
TTL	5 seconds for each A answer
Query-response time delta	Approximately 0.454833 seconds / 454.833 ms
Subsequent connection correlation	ssl.gstatic.com resolved to 142.251.216.35; the client subsequently initiated a TCP connection to 142.251.216.35:443, approximately 2.418 ms later
Normal variations observed	Multiple A answers for one domain; multiple browser-generated DNS queries for Google, Mozilla, fonts, ads, APIs and supporting services; several DNS query types were observed; the optional SMTP evidence also showed a CNAME chain from mail.patriots.in to patriots.in before resolving to 74.53.140.153

Limitations and Normal Variations

One limitation I encountered happened during the browser correlation stage. The original browser DNS capture was created using a port 53 capture filter, so it contained DNS packets but did not retain the later TCP connection packets that were needed for the correlation exercise. Because of this, I kept the original DNS-only capture for the browser inventory and created a separate browser_correlation.pcapng without the port-53-only restriction. This allowed me to preserve both the DNS answers and the later TCP SYN connections in the same capture and complete the correlation correctly.

I also observed several normal DNS variations during the practical. A single domain could return more than one IP address, as seen with example.com, which returned two IPv4 addresses. Browser activity generated many more DNS queries than the controlled dig command because the Google page and the browser contacted additional services for fonts, advertisements, APIs, browser features and other supporting resources. Different query types were also present in the browser inventory.

The optional SMTP evidence showed another normal DNS variation. The query for mail.patriots.in did not resolve directly to an IP address. Instead, it used a CNAME relationship to patriots.in, which then resolved to 74.53.140.153. I also observed different TTL values between the CNAME, A record and authoritative name-server records. These differences showed me that normal DNS responses can contain several record types and do not always follow a simple one-domain-to-one-IP pattern.

Conclusion

From this practical, I was able to build a clear baseline of normal DNS behaviour using evidence I generated and analysed myself. I identified my configured resolver, queried several DNS record types, captured a fresh request and response, and matched them using the transaction ID and endpoint information. I also calculated the response delay and confirmed that the example.com query completed successfully with a NOERROR response.

The browser analysis helped me understand that real web activity creates much more DNS traffic than a single command-line query. My strongest correlation result showed that ssl.gstatic.com resolved to 142.251.216.35 and that my system then initiated a TCP connection to the same IP on port 443 about 2.418 ms later. This gave me direct evidence linking DNS resolution with subsequent network communication.

Overall, the lab improved my understanding of how DNS evidence can be examined during a digital-forensics investigation. I was able to see how resolver information, DNS record types, transaction IDs, TTL values, response timing and later connections can be combined to explain what a system was trying to reach and what happened immediately after name resolution.

Screenshots

Fig 1.0 Lab folder structure.

Fig 1.1 Checking the required tools.

Fig 1.2 Checking the required tools page 1.

Fig 1.3 Configured DNS Resolver.

Fig 1.4 Evidence preparation / hash verification.

Fig 1.5 Dig A outputs.”.

Fig 1.6 Dig AAAA outputs.”.

Fig 1.7 Dig MX outputs.”.

Fig 1.8 Dig NS outputs.”.

Fig 1.9 Dig example_short.txt outputs.”.

Fig 2. Confirming your active Ethernet.

Fig 2.1 Fresh DNS Capture Hash.

Fig 2.2 Extracting DNS Query and Response Fields.

Fig 2.3 Analyzing DNS Generated by a Browser.

Fig 2.4 Analyzing DNS Generated by a Browser page 1.

Fig 2.5 Analyzing DNS Generated by a Browser page 2.

Fig 2.6 Analyzing DNS Generated by a Browser page 3.

Fig 2.7 Analyzing DNS Generated by a Browser page 4.

Fig 2.8 Correlate DNS with Subsequent Connections.

Fig 2.9 Correlate DNS with Subsequent Connections page 1.

Fig 3.0 Correlate DNS with Subsequent Connections page 2.

Fig 3.0.1 Correlate DNS with Subsequent Connections web connection.

Fig 3.1 TCP Destination Correlation.

Fig 3.2 TCP Destination Correlation page 1.

Fig 3.3 SMTP DNS Correlation.

References

ICDFA Materials.